

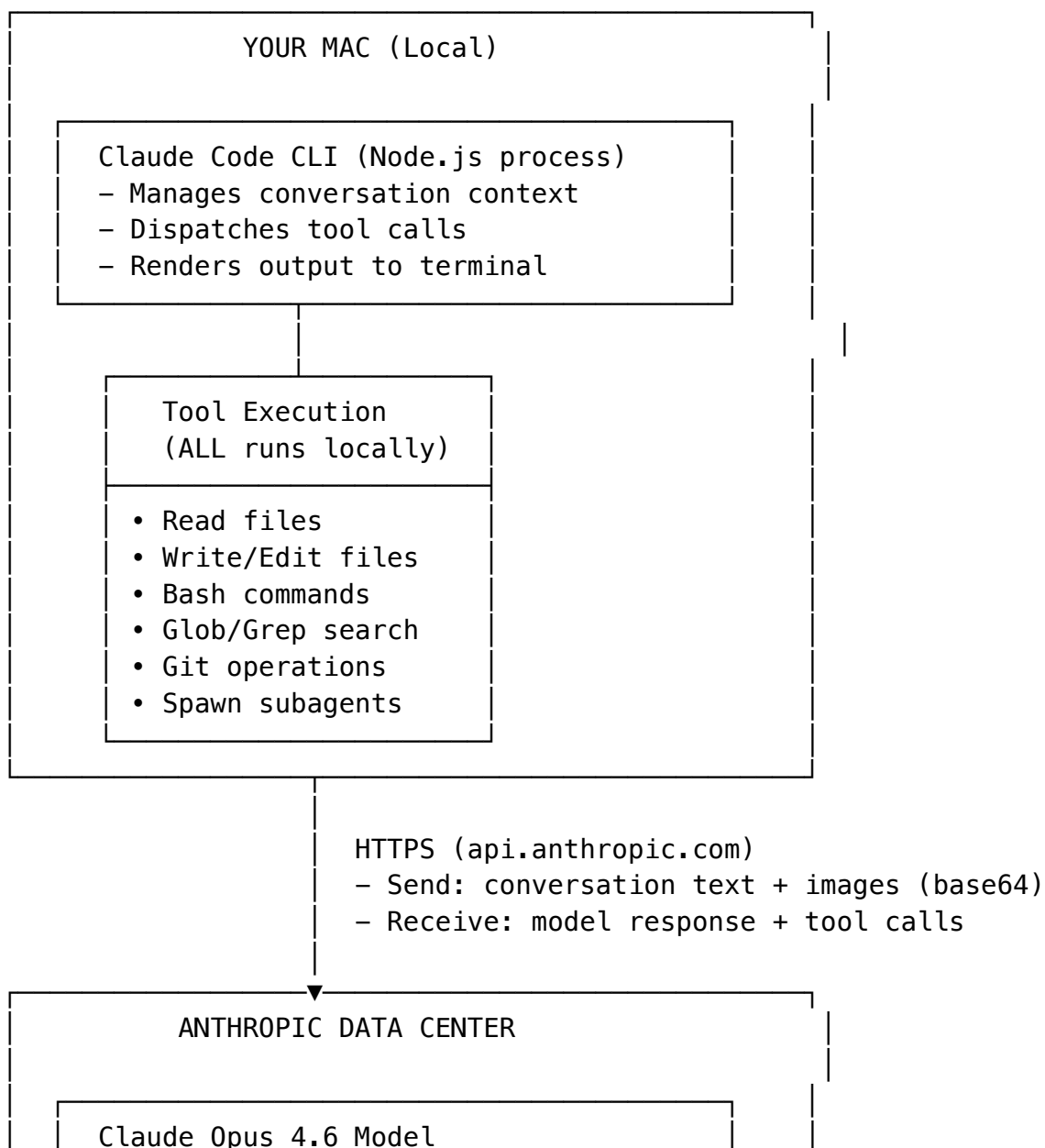
Claude Code Architecture – How It Actually Works

Date: 2026-02-24

Machine: MacBook Pro M3 Max (Mac15,10), 36 GB RAM

Overview

Claude Code is a CLI tool that runs **locally on your machine** as a Node.js process. It orchestrates between local tool execution and remote LLM inference via Anthropic's API.



- Processes text and images
- Generates analysis/code/responses
- Decides which tools to call next
- Returns structured tool call JSON

Rate limits enforced per account:

- Pro Max: flat rate, shared across all agents
- API: per-token billing

The Request-Response Cycle

Every “turn” in a Claude Code conversation follows this loop:

1. User types message (or agent continues autonomously)



2. Local CLI packages:
 - System prompt + instructions
 - Conversation history
 - Any images (base64-encoded PNGs)
 - Available tool definitions



3. HTTPS POST to `api.anthropic.com/v1/messages`
(payload can be 1-10 MB with images)



4. Anthropic servers run inference
(Claude “thinks” – reads images, analyzes context)



5. Response streams back:
 - Text blocks (shown to user)
 - Tool use blocks (executed locally)



6. CLI executes tool calls LOCALLY:
 - `Read("/path/to/file")` → reads from your disk
 - `Write("/path", content)` → writes to your disk
 - `Bash("git status")` → runs on your machine
 - `Grep("pattern")` → searches your files



7. Tool results sent back to API as next turn



8. Loop back to step 4 (model sees tool results, decides next action)

Key insight: The model never directly touches your filesystem. It can only REQUEST tool calls, which the local CLI executes. Every tool call goes through a permission check before execution.

What Runs Where

On Your Mac (Local)

- **Process management** – CLI process, subagent processes
- **All file I/O** – reading charts, writing reports, editing code
- **All shell commands** – git, python, npm, docker, etc.
- **Image encoding** – chart PNGs converted to base64 before sending
- **Permission enforcement** – user approves/denies tool calls
- **Context management** – conversation history, compression

On Anthropic's Servers (Remote)

- **LLM inference** – the actual “thinking” / analysis / code generation
- **Vision processing** – reading chart images, screenshots
- **Tool call decisions** – model decides WHICH tools to call with WHAT parameters
- **Rate limiting** – enforced per account across all concurrent sessions

Nothing Runs On

- No cloud VMs spun up for you
 - No containers or sandboxes (unless you configure them)
 - No persistent state on Anthropic's side between sessions
-

Resource Usage on Your Mac

Resource	Usage	Notes
CPU	Minimal (~2-5%)	Just process management and I/O
RAM	~200-500 MB per agent	Node.js process + conversation context
Disk	Read/write at full SSD speed	Chart PNGs, reports, code files

Resource	Usage	Notes
Network	1–10 MB per API call	Images are the bulk of the payload
Battery	Low drain	Most work is waiting for API responses

With 5 concurrent agents: ~1–2.5 GB RAM, minimal CPU, network is the bottleneck.

Billing Models

Pro Max Subscription (\$100–200/month flat)

- Unlimited* conversations via claude.ai and Claude Code CLI
- Rate limited (requests per minute, not tokens)
- All agents share the same rate limit pool
- **Best for:** High-volume analysis like our chart pipeline
- **Practical limit:** ~5 concurrent agents before rate limiting

API Key (Pay-per-token)

- Billed per input/output token
- Opus: ~\$15/M input, \$75/M output tokens
- Images: ~1,000 tokens per chart image
- **Best for:** Production pipelines, other models (Sonnet/Haiku cheaper)
- **Our cost:** ~\$0.15–0.20 per ticker (3 charts × Opus)

Hybrid Approach (What We Use)

- **Pro Max** for daily interactive analysis (Task agents reading charts)
- **API** reserved for production pipeline with model flexibility
- Saves ~\$150–200/day vs running everything through API